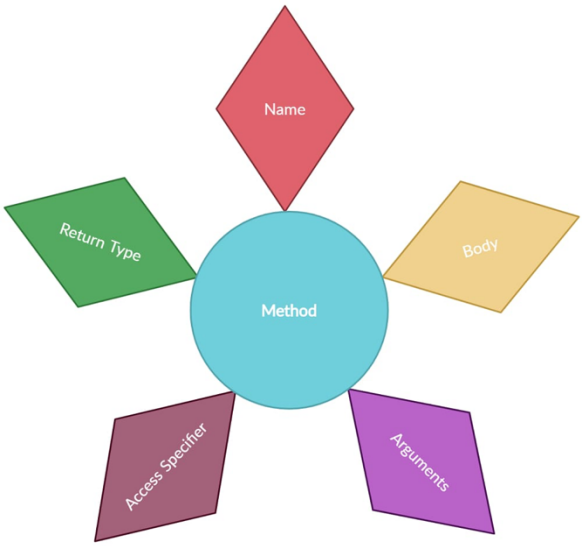
METHODS

Methods- Introduction

In simple terms, a method is a block of code that performs a specific function. Methods provide for easy modification and code reusability. We write a method once and use it many times.

We do not require to write code again and again. It also provides the **easy modification** and **readability** of code, just by adding or removing a chunk of code. The method is executed only when we call or invoke it.



Let us try to understand Methods in a simpler way, consider a scenario where you have a common task that is repeatedly performed in a program. There are two ways to handle this scenario while programming.

- You can write the same common program logic again and again and complete the program.
- You can separate the common logic and write it in a Method once and just call that method again and again.

Declaration and Components of a Method

First, a method consists of **SIX** components:

```
access modifier  return type  method identifier          parameter list                                  exception list
public String getName(String firstName, String lastName) throws RuntimeException {
    return firstName + " " + lastName;  body
}
```

- **Access modifier:** Optionally we can specify from wherein the code one can access the method.
- **Return type:** The type of the value returned by the method, if any.
- **Method identifier:** The name we give to the method.
- **Parameter list**: An optional comma-separated list of inputs for the method.
- **Exception list:** An optional list of exceptions the method can throw.
- **Body:** Definition of the logic (can be empty).

Access Modifier

The access modifier allows us to specify which objects can have access to the method. There are four possible access modifiers: public, protected, private, and default (also called package-private).

A method can also include the static keyword before or after the access modifier. This means that the method belongs to the class and not to the instances, and therefore, we can call the method without creating an instance of the class. Methods without the static keyword are known as instance methods and may only be invoked on an instance of the class.

Regarding performance, a static method will be loaded into memory just once – during class loading – and are thus more memory-efficient.

Return Type

Methods can return data to the code where they have been called from. A method can return a primitive value or an object reference, or it can return nothing if we use the *void* keyword as the return type.

If we declare a return type, then we have to specify a *return* statement in the method body. Once the *return* statement has been executed, the execution of the method body will be finished and if there are more statements, these won't be processed.

On the other hand, a *void* method doesn't return any value and, thus, does not have a *return* statement.

Parameter List

We can specify input values for a method in its parameter list, which is enclosed in parentheses. A method can have anywhere from 0 to 255 parameters that are delimited by commas. A parameter can be an Object, Primitives or an Enumeration. We can use Java annotations at the method parameter level.

Exception List

We can specify which exceptions are thrown by a method by using the *throws* clause. In the case of a checked exception either we must enclose the code in a try-catch clause or we must provide a *throws* clause in the method signature.

Method Body

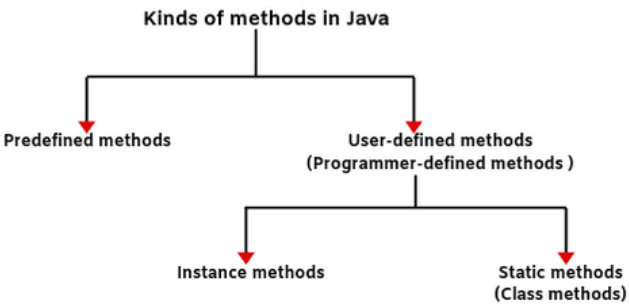
The last part of a Java method is the method body, which contains the logic we want to execute. In the method body, we can write as many lines of code as we want — or none at all in the case of *static* methods. If our method declares a return type, then the method body must contain a return statement.

Methods Signature

As per its definition, a method signature is comprised of only two components — the method's identifier and parameter list. Remember other components are not part of signature.

The method identifier can be any identifier. However, if we follow common Java coding conventions, the method identifier should be a verb in lowercase that can be followed by adjectives and/or nouns.

Types of Method



There are **TWO** types of methods in Java:

- Predefined Method
- User-defined Method

Predefined Methods

In Java, predefined methods are the method that is already defined in the Java class libraries is known as **predefined methods.** It is also known as the **standard library method** or **built-in method**. We can directly use these methods just by calling them in the program at any point. Some pre-defined methods are **length(), equals(), compareTo(), sqrt(),** etc. When we call any of the predefined methods in our program, a series of codes related to the corresponding method runs in the background that is already stored in the library. Each and every predefined method is defined inside a class. Such as **print()** method is defined in the **java.io.PrintStream** class. It prints the statement that we write inside the method. For example, **print("Java")**, it prints Java on the console.

How to call Pre-Defined Methods: We should have an idea about the data type of the method, what it will return and what is it's type like either it is Instance Method or Static Method.

User-Defined Methods

The method written by the user or programmer is known as **a user-defined** method. These methods are modified according to the requirement. We create it declaring it using the six or less components.

User Defined Methods are of TWO Types:

- Static Methods
- Instance Methods

Static Methods

A method that has static keyword is known as static method. In other words, a method that belongs to a class rather than an instance of a class is known as a static method. We can also create a static method by using the keyword **static** before the method name.

How to call Static Methods: The main advantage of a static method is that we can call it without creating an object. To call a static method, we use dot operator with the class name.

It can access static data members and also change the value of it. It is used to create an instance method. It is invoked by using the class name. The best example of a static method is the **main()** method.

Instance Methods

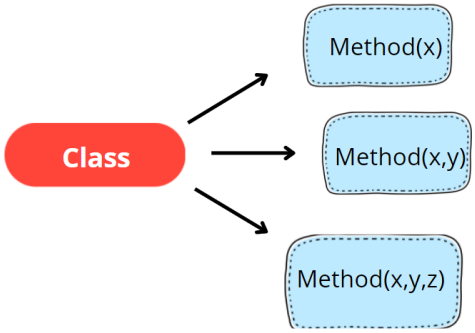
The method of the class is known as an **instance method**. It is a **non-static** method defined in the class.

How to call Instance Methods: Before calling or invoking the instance method, it is necessary to create an object of its class. Second, call the method with a reference variable using the dot operator.

We will discuss more about it in OOP's.

Methods Overloading

Java allows us to have **two or more methods with the same identifier but different parameter list — different method signatures**. In this case, we say that **the method is overloaded**.



Method overloading is useful for cases like the one in the example, where we can have a method implementing a simplified version of the same functionality.

Finally, a good design habit is to ensure that Overloaded Methods behave in a similar manner. Otherwise, the code will be confusing if a method with the same identifier behaves in a different way.

For Example: We have to do the sum of values but sometimes I have to take two values as input but sometime I want to take three values as input. That means functionality is same but the number of inputs changes. Here method overloading comes in action.

Simply put, we can implement method overloading in two different ways:

- Implementing two or more methods that have the **same name but take different numbers of arguments.**
- Implementing two or more methods that have the **same name but take arguments of different data types.**

It's worth noting, however, that it's not possible to have two method implementations that differ only in their return types.

One neat feature provided by method overloading is the **so-called type promotion**.

In simple terms, one given type is implicitly promoted to another one when there's no matching between the types of the arguments passed to the overloaded method and a specific method implementation.

The ability to associate a specific method call to the method's body is known as **binding**.

In the case of method overloading, the binding is performed statically at compile time, hence it's called **static binding.**

The compiler can effectively set the binding at compile time by simply checking the methods' signatures.

Memory Management of Methods

Declaration of a method is done at the compile time. Calling of a function is done at runtime.

A function is responsible for doing **TWO** task only:

- Calling some other function.
- Execution of some programme.

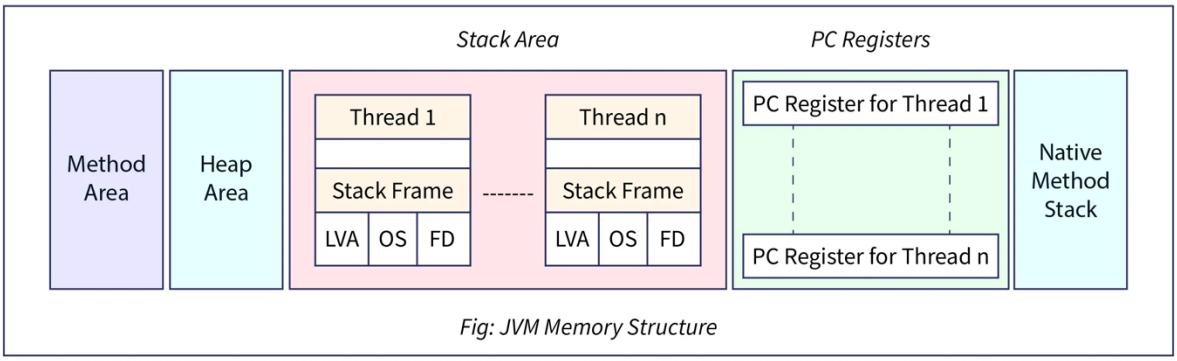


Fig: JVM Memory Structure

At runtime the function remains in the memory unless it returns the value to the place where it was called. First the main method is called so it gets inside the memory then following it other functions get inside the memory. When a method return it value the it comes out of the memory following LIFO. Lastly when the main method executes it comes out of the stack the programme ends.

The stack memory is used for storing method frames. Each time a method is called, a new frame is created on the stack to hold the method's parameters, local variables, and intermediate computation values. The stack memory follows the Last In, First Out (LIFO) principle, meaning the most recently called method's frame is at the top of the stack, and it is removed when the method completes its execution. Stack memory is generally smaller in size and has a fast allocation and deallocation mechanism.

Passing in Java

The two most prevalent modes of passing arguments to methods are “passing-by-value” and “passing-by-reference”. Different programming languages use these concepts in different ways. **As far as Java is concerned, everything is strictly** Pass-by-Value.

Let's start with some of the different mechanisms for passing parameters to functions:

- value
- reference
- result
- value-result
- name

The two most common mechanisms in modern programming languages are “Pass-by-Value” and “Pass-by-Reference”.

Pass By Value

When a parameter is pass-by-value, the caller and the callee method operate on two different variables which are copies of each other. Any changes to one variable don't modify the other.

It means that while calling a method, **parameters passed to the callee method will be clones of original parameters.** Any modification done in callee method will have no effect on the original parameters in caller method.

Pass By Reference

When a parameter is pass-by-reference, the caller and the callee operate on the same object. It means that when a variable is pass-by-reference, **the unique identifier of the object is sent to the method.** Any changes to the parameter's instance members will result in that change being made to the original value.

Parameter Passing Types

The fundamental concepts in any programming language are "values" and "references". In Java, **Primitive variables store the actual values, whereas Non-Primitives store the reference variables which point to the addresses of the objects they're referring to.** Both values and references are stored in the stack memory.

Arguments in Java are always passed-by-value. During method invocation, a copy of each argument, whether it's a value or reference, is created in stack memory which is then passed to the method.

In case of primitives, the value is simply copied inside stack memory which is then passed to the callee method; in case of non-primitives, a reference in stack memory points to the actual data which resides in the heap. When we pass an object, the reference in stack memory is copied and the new reference is passed to the method.

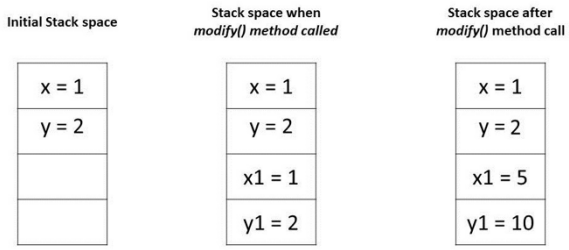
Passing Primitive Types

The Java Programming Language features eight primitive types. Primitive variables are directly stored in stack memory. Whenever any variable of primitive data type is passed as an argument, the actual parameters are copied to formal arguments and these formal arguments accumulate their own space in stack memory.

The lifespan of these formal parameters lasts only as long as that method is running, and upon returning, these formal arguments are cleared away from the stack and are discarded.

Let's try to understand the assertions in the above program by analysing how these values are stored in memory:

1. The variables **"*x*" and "*y*"** in the main method are primitive types and their values are directly stored in the stack memory.
2. When we call method ***modify()***, an exact copy for each of these variables is created and stored at a different location in the stack memory.
3. Any modification to these copies affects only them and leaves the original variables unaltered.



Passing Non-Primitive Types

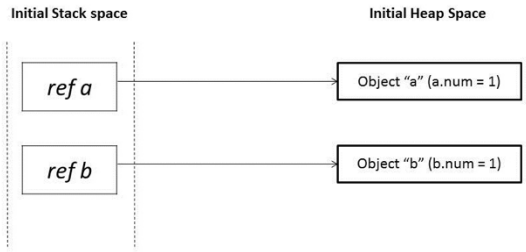
In Java, all objects are dynamically stored in Heap space under the hood. These objects are referred from references called reference variables.

A Java object, in contrast to Primitives, is stored in two stages. The reference variables are stored in stack memory and the object that they're referring to, are stored in a Heap memory.

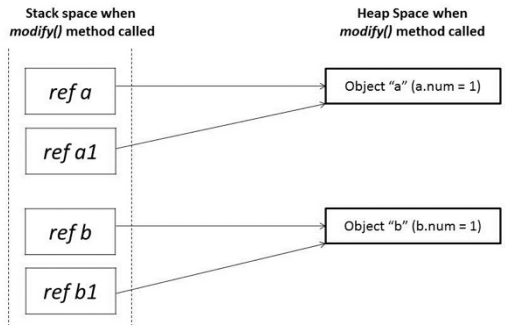
Whenever an object is passed as an argument, an exact copy of the reference variable is created which points to the same location of the object in heap memory as the original reference variable.

As a result of this, whenever we make any change in the same object in the method, that change is reflected in the original object. However, if we allocate a new object to the passed reference variable, then it won't be reflected in the original object.

Let's analyse the assertions in the above program. We have passed **objects *a* and *b* in *modify()*** method that has the same value 1. Initially, these object references are pointing to two distinct object locations in a heap space.

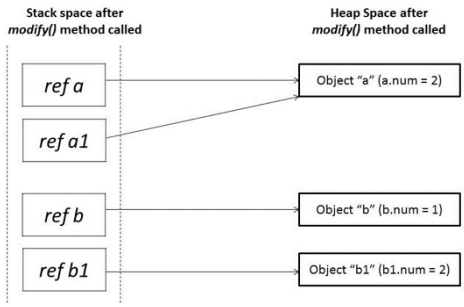


When these references *a* and *b* are passed in the *modify()* method, it creates mirror copies of those references ***a1*** **and *b1* which point to the same old objects.**



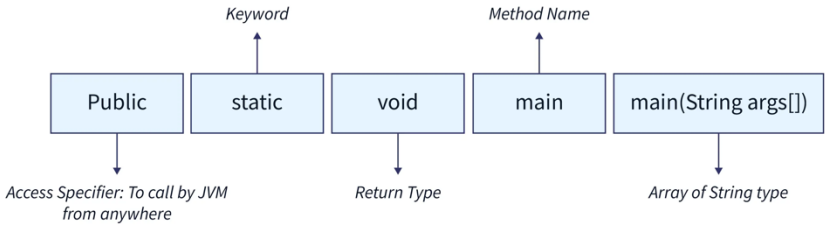
In the ***modify()*** method, when we modify reference ***a1***, it changes the original object. However, for a reference ***b1,*** we have assigned a **new object**. So it's now pointing to a new object in heap memory.

Any change made to ***b1*** **will not reflect anything in the original object.**



Main Method

The main() is the starting point for JVM to start execution of a Java program. Without the main() method, JVM will not execute the program. The syntax of the main() method is.



https://replit.com/@ManuChaitanya/methods#Main.java

public

- It is an access modifier of the main() method that specifies by whom and from where the method can be accessed. We create the main() method with a public access specifier so that it can be executed by any program (globally available). Since it is not a part of the current class, it is made public so that JVM can call it from outside the class. Therefore, it is necessary to make the main() method public.
- private, protected, and public are just a few of the access modifiers that Java specifies. Any method or variable can be accessed from outside the class if it has been declared as public. Since the main method is declared public in Java, JVM can easily access and execute. If we declare the main() method as non-public, it will throw the following error:

```
Error: Main method not found in class Java, please define the main method as:
   public static void main(String[] args)
or a JavaFX application class must extend javafx.application.Application
```

static

- The term static is used in the main() method to make it a class-related method. The main() method is declared static so that JVM can call it without creating an instance of the class containing the main() method. We must declare the main() function static as no class object is present when the java runtime starts. JVM can then load the class into the main memory and invoke the main() method.
- Initiating the main() method as static also prevents the JVM from wasting memory that would have been utilized by the object specified only for invoking the main() method. As a result, if we specify the main() method as non-static, JVM will be unable to invoke it and will throw the desired error:

```
Error: Main method is not static in class Java, please define the main method as:
   public static void main(String[] args)
```

void

- Each method in java has a different return type, such as Integer, Double, Boolean, String, etc. A Java main() function does not return anything to keep things simple and has the return type void.
- After executing the main() method, the program will be terminated, and returning anything from the main() method is useless because the JVM cannot do anything with the return value. The following error will be thrown if we do not provide the main() method with a void return type:

```
/tmp/P318g6qcLQ/Java.java:7: error: invalid method declaration; return type required
    public static main(String[] args) {
                  ^
```

main()

It is considered the name of the main() method in Java and isn't a keyword. The JVM looks for this identifier as the starting point of the Java program. The method should be declared as main, and we cannot modify it. The following error will be thrown if we try to change the name of the method():

```
Error: Main method not found in class Public, please define the main method as:
   public static void main(String[] args)
or a JavaFX application class must extend javafx.application.Application
```

String[] args or String args[]

- It is an array of type *java.lang.string* class that accepts a single string array argument which is often known as **Java command-line arguments**. The args is the name of the String array, but it is not fixed as the user may specify any other name in its place.
- In addition to the above-mentioned signature, you can call the main method using either public static void main(String args[]) or public static void main(String... args), as both are acceptable. The main method gets invoked if the main method's formal parameter matches that of an array of Strings.

Reasons behind Defining the main() Method as Static

It is not possible to invoke a function without constructing an instance of its class, as there is no object of a class when the JVM starts. The main() function is declared static such that the class can be loaded by JVM into the main memory.

- Every Java program begins with the main() function. The main() method is essential because the compiler begins executing a program from the main() function only.
- The JVM should instantiate the class if the main() function is non-static.
- JVM (Java Virtual Machine) can simply invoke static methods without generating an instance of the class by utilizing only the class name.
- The main() function must be declared static, public, and must have a void return type. The JVM will be unable to run the java program and will throw an error if we do not declare it as static and public or if we do not provide it with a void return type.